

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Engenharia de Software

Teste de Software

Laboratório de Test Smells

Identificação e refatoração de maus cheiros em testes automatizados

Guilherme Gomes de Brites

Matrícula: 808721

Belo Horizonte

2026

1. Análise de Smells

A suíte original (`userService.smelly.test.js`) passa no `npm test`, mas isso não quer dizer que ela seja boa. Lendo teste por teste, encontrei vários problemas. Abaixo estão os três que achei mais importantes.

1.1 Lógica Condicional no Teste

O teste `deve desativar usuários se eles não forem administradores` usa um `for` e um `if/else` para decidir quais asserções rodar. Isso é um mau cheiro porque o teste passa a ter lógica própria: ele mesmo pode ter bugs, e ninguém testa o teste. Além disso, ele mistura duas regras diferentes (usuário comum pode ser desativado, admin não pode) num lugar só, com um nome que só fala de uma delas.

Risco: se a lista mudar ou um dos ramos nunca for executado, algumas asserções simplesmente não rodam e o teste continua verde. Quando ele falha, também é difícil saber qual das duas regras quebrou.

1.2 Falha Silenciosa com try/catch

O teste `deve falhar ao criar usuário menor de idade` coloca o `expect` apenas dentro do `catch`. Se a exceção for lançada, tudo certo. Mas se alguém apagar a validação de idade do `createUser`, nenhum erro acontece, o `catch` não roda, nenhum `expect` é executado e o teste passa.

Risco: é o mais perigoso dos três, porque gera um falso positivo. O teste existe justamente para pegar essa regressão e não pega. Dá uma falsa sensação de segurança.

1.3 Teste Frágil

O teste `deve gerar um relatório de usuários formatado` compara a linha exata do relatório, com espaços, vírgulas e quebra de linha (`ID: ..., Nome: Alice, Status: ativo\n`). O próprio código do serviço avisa que esse formato pode mudar.

Risco: qualquer ajuste de layout, como trocar a ordem dos campos ou adicionar um espaço, quebra o teste sem que o comportamento tenha mudado. Com o tempo, a equipe se acostuma a "consertar o teste" sem pensar, e ele deixa de ajudar.

2. Processo de Refatoração

Escolhi o teste de desativação de usuários, porque ele junta dois problemas: lógica condicional e duas regras de negócio num teste só.

ANTES (test/userService.smelly.test.js)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  // O teste tem um loop e um if, tornando-o complexo e menos claro.
  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      // Este expect só roda para o usuário comum.
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      // E este só roda para o admin.
      expect(resultado).toBe(false);
    }
  }
});
```

DEPOIS (tests/userService.clean.test.js)

```
test('deve desativar um usuário comum', () => {
  // Arrange
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

  // Act
  const resultado = userService.deactivateUser(usuarioComum.id);

  // Assert
  expect(resultado).toBe(true);
  expect(userService.getUserById(usuarioComum.id).status).toBe('inativo');
});

test('não deve desativar um usuário administrador', () => {
  // Arrange
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  // Act
  const resultado = userService.deactivateUser(usuarioAdmin.id);

  // Assert
  expect(resultado).toBe(false);
  expect(userService.getUserById(usuarioAdmin.id).status).toBe('ativo');
});
```

As decisões que tomei foram estas:

- **Um teste por regra.** Separei o teste em dois, um para o usuário comum e outro para o admin. Agora, se um falhar, o nome já diz qual regra quebrou.
- **Sem for e sem if.** Cada teste é uma sequência reta de passos. Todas as asserções sempre rodam, então não existe mais o risco de um `expect` ser pulado.
- **Padrão AAA.** Cada teste tem as três partes marcadas: preparar os dados (Arrange), chamar o método (Act) e verificar o resultado (Assert). Fica fácil ler de cima para baixo.
- **Nomes claros.** Os nomes descrevem o comportamento esperado, em português, sem termos vagos.
- **Verificação mais completa.** No caso do admin, além do retorno `false`, conferi que o status continua `ativo`, que é o que realmente importa para a regra.

Nos outros testes, segui a mesma linha: o teste de criação e busca virou testes menores e focados, o `try/catch` foi trocado por `expect(...).toThrow(...)` (que falha se o erro não for lançado), o teste do relatório passou a verificar só se os dados aparecem (`toContain`) e o teste que estava com `skip` foi implementado.

3. Relatório da Ferramenta

Configurei o ESLint com o plugin `eslint-plugin-jest` e rodei `npx eslint .` no projeto. A primeira execução, ainda só com o arquivo original, mostrou o seguinte:



```
test-smelly : zsh

test-smelly % npx eslint .

/Users/guilhermedebrites/github/faculdade/test-smelly/test/userService.smelly.test.js
 44:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning Tests should not be skipped          jest/no-disabled-tests
 77:3  warning Test has no assertions          jest/expect-expect

× 6 problems (4 errors, 2 warnings)
```

Figura 1: resultado da primeira execução do ESLint (4 erros e 2 avisos).

A ferramenta encontrou sozinha três dos smells que eu tinha anotado na análise manual. A regra `jest/no-conditional-expect` apontou os `expect` dentro do `if/else` (linhas 44, 46 e 49) e dentro do `catch` (linha 73). As regras `jest/no-disabled-tests` e `jest/expect-expect` apontaram o teste com `skip` e sem asserções (linha 77). Tudo isso em menos de um segundo e sem precisar ler o código.

Por outro lado, o ESLint não pegou o Teste Frágil. Faz sentido: ele procura padrões no código, e comparar uma string não é errado em si. Para perceber que o teste está preso à formatação, é preciso entender a intenção dele. Por isso a ferramenta não substitui a revisão humana, ela complementa. Depois da refatoração, rodei o ESLint de novo e o arquivo `userService.clean.test.js` ficou com zero erros e zero avisos. O `npm test` passou com todas as suítes (12 testes passando e 1 pulado, que é o `skip` do arquivo original).

4. Conclusão

Este trabalho mostrou que um teste que passa não é, necessariamente, um bom teste. A suíte original estava toda verde, mas tinha um teste que nunca ia falhar, outro que quebraria por qualquer mudança de layout e outro com lógica difícil de acompanhar. Testes assim custam caro: geram confiança falsa e dão trabalho na manutenção.

Escrever testes limpos, com um comportamento por teste, padrão AAA e nomes claros, faz com que eles funcionem também como documentação do sistema. Quando algo quebra, o problema aparece rápido e no lugar certo. Já as ferramentas de análise estática, como o ESLint, ajudam a manter esse padrão sem depender só da atenção de cada pessoa, principalmente se rodarem automaticamente no dia a dia ou no pipeline. Juntando as duas coisas, o projeto fica mais fácil de evoluir e os testes continuam sendo úteis com o passar do tempo.